

Zadanie z programowania w C++

Treść zadania

Napisz program w języku C++, który implementuje klasę `Camera` oraz wymagane klasy pomocnicze. Klasa `Camera` reprezentuje kamerę w trójwymiarowej scenie.

Wymagania dotyczące implementacji

Zaimplementuj następujące klasy:

- **Klasa `vec3`** (dwa pliki: `vec3.h` (deklaracje) oraz `vec3.cpp` (implementacja))
 - Klasa nie ma być szablonem.
 - Posiada trzy składowe publiczne reprezentujące współrzędne wektora 3D.
 - Konstruktor bezparametrowy inicjalizujący wszystkie współrzędne na zero.
 - Konstruktor trójparametrowy inicjalizujący współrzędne wartościami podanymi w argumentach.
 - Funkcja składowa `normalize`, która normalizuje wektor.
- **Klasa `mat4`** (dwa pliki: `mat4.h` oraz `mat4.cpp`)
 - Klasa nie ma być szablonem.
 - Odpowiada za przechowywanie macierzy 4x4 i operacje na niej.
 - Konstruktor bezparametrowy inicjalizujący wszystkie współrzędne na zero
 - Posiada przeładowany `operator+`.
- **Klasa `Camera`** (dwa pliki: `Camera.h` oraz `Camera.cpp`)
 - Prywatne pola:
 - * `m_position` - pozycja kamery w przestrzeni $\mathbf{R}^3$,
 - * `m_up` - wektor kierunku do góry kamery,
 - * `m_front` - wektor wskazujący przód kamery,
 - * `float m_pitch` - kąt nachylenia,
 - * `float m_yaw` - kąt obrotu w poziomie,
 - * `float m_fov` - pole widzenia kamery.
 - * `GLFWwindow* m_window` - wskaźnik do okna aplikacji.
 - Funkcje składowe:
 - * Napisz konstruktor, który jako argument przyjmuje wskaźnik do okna aplikacji `GLFWwindow*`. Konstruktor inicjalizuje wszystkie zmienne/pola w klasie (przyjąć jakieś losowe wartości).

- * Napisz destruktor, który nic nie robi.
- * Napisz funkcję o nazwie `ProcessMouseScroll`, która nie zwraca niczego oraz przyjmuje jeden argument będący liczbą zmiennoprzecinkową o nazwie `yOffset`. Zanegowana wartość parametru `yOffset` jest dodawana do zmiennej `m_fov`. Należy zadbać, aby wartość `m_fov` zawsze była z przedziału $[10, 45]$.
- * Napisz funkcję o nazwie `ProcessMouseMove`, która nie zwraca niczego oraz przyjmuje dwa argumenty typu float: `deltaX`, `deltaY`. Wartość `deltaX` jest dodawana do zmiennej `m_yaw`, a argument `deltaY` jest dodawany do `m_pitch`. Należy zadbać, aby wartość `m_fov` zawsze była z przedziału $[-89, 89]$. Następnie utwórz dwie zmienne o nazwie `y` oraz `p`, które będą zawierać wartości `m_pitch` i `m_yaw` wyrażone w radianach. $y = x \cdot \pi / 180$. Dalej stwórz wektor o nazwie `direction`, którego współrzędne wyrażają się następująco:

$$x = \cos y \cdot \cos p, \quad (1)$$

$$y = \sin p, \quad (2)$$

$$z = \sin y \cdot \cos p. \quad (3)$$

Na koniec znormalizuj wektor `direction` i przypisz jego zawartość do zmiennej `m_front`.

- * Napisz funkcję o nazwie `ProcessKeyboardInput`, która nie zwraca niczego oraz przyjmuje jeden argument będący liczbą zmiennoprzecinkową o nazwie `deltaTime`. Ciało funkcji pozostaw puste.
- * Napisz funkcję o nazwie `GetViewMatrix`, która zwraca obiekt klasy `mat4`. Funkcja zwraca pustą macierz.
- * Napisz funkcję o nazwie `GetPosition`. Funkcja zwraca aktualną pozycję kamery.
- * Napisz funkcję o nazwie `GetFov`. Funkcja zwraca aktualną wartość pola widzenia kamery.

W funkcji `int main()` utwórz obiekt typu `Camera` oraz obiekty typu: `mat4`, `vec3`.